

Reflexion Woche 7

REST-API Implementierung

- **Wie war die Erfahrung, die erste API von Grund auf zu bauen? Was war schwierig?**

- Das Aufsetzen einer eigenen REST-API von Grund auf war eine sehr lehrreiche Erfahrung, da wir die zugrundeliegende HTTP-Logik nun viel besser verstehen. Besonders schwierig war anfangs das saubere Verknüpfen der verschiedenen Routen-Dateien sowie die korrekte Handhabung von asynchronen Zuständen und ID-Pfaden im Terminal.

- **Welche CRUD-Operationen waren am einfachsten zu implementieren?**

- Die `GET`-Operationen (zum Abrufen aller oder einzelner Tasks) sowie der `POST`-Endpunkt zum Erstellen neuer Aufgaben waren am einfachsten umzusetzen, da sie einer standardisierten Struktur folgen. Hier mussten wir lediglich Daten auslesen oder ein neues Objekt an das bestehende Array anhängen, ohne komplexe Filterregeln anzuwenden.

- **Warum ist der `PATCH /transition` Endpoint besonders wichtig für Kanban?**

- Dieser Endpoint ist essenziell, weil er im Gegensatz zu einem plumpen Update-Befehl gezielt den Zustandswechsel (die Transition) einer Aufgabe steuert. Er erlaubt es uns, eine automatisierte Validierungs-Middleware zwischenzuschalten, die illegale Status-Sprünge – wie das Überspringen der Testphase – serverseitig strikt blockiert.

Kanban-Board und WIP-Limits

- **Was ist der Sinn von WIP-Limits? Wie verhindert es Chaos im Team?**

- Der Sinn von WIP-Limits (Work-in-Progress) ist es, die Menge parallel begonnener Aufgaben gezielt zu begrenzen, um Multitasking und Engpässe zu vermeiden. Es verhindert Chaos, indem es das Team dazu zwingt, bestehende Aufgaben im Sinne von „Stop Starting – Start Finishing“ erst gemeinsam abzuschließen, bevor neue Arbeit angefangen wird.

- **Wenn "In Progress" ein WIP-Limit von 3 hat – was passiert, wenn ein Team-Member versucht, eine 4. Task zu starten?**

- In einem physischen oder strengen digitalen System wird das Verschieben der vierten Task blockiert, um das Limit zu wahren. Übertragen auf unsere API führt genau dieser Versuch zu einem kontrollierten Fehler (wie ein HTTP `400 Bad`

`Request`), der das Team visuell oder programmatisch darauf hinweist, dass zuerst ein Engpass gelöst werden muss.

- **Wie unterscheidet sich Kanban-Thinking von klassischem Wasserfall?**

- Während der klassische Wasserfall auf starren, sequenziellen Phasen und einer langfristigen Vorab-Planung basiert, setzt Kanban auf einen kontinuierlichen, flexiblen Wertefluss (Pull-Prinzip). Kanban erlaubt es uns, jederzeit auf Veränderungen zu reagieren und Aufgaben inkrementell zu bearbeiten, anstatt alles in einem einzigen, großen Release am Ende auszuliefern.

Vergleich API und Jira Board

- **Wie hängen die Task-Statuses in Ihrer API mit den Jira-Spalten zusammen?**

- Die Status-Werte in unserer API (wie `backlog` , `todo` , `in_progress` , `done`) bilden das exakte Spiegelbild der Spaltenstruktur auf unserem Jira-Kanban-Board ab. Jede Spalte im Frontend (Jira) korrespondiert somit eins zu eins mit einem gültigen Zustand in der Backend-Datenstruktur unserer API.

- **Wie könnten Sie Ihre API mit Jira verbinden (API und Jira Webhook)?**

- Wir könnten in Jira einen sogenannten Webhook einrichten, der bei jedem Verschieben einer Karte automatisch einen HTTP-Informationenruf an unsere API absetzt. Unsere API müsste dann einen speziellen Empfänger-Endpunkt (z. B. `POST /api/jira-webhook`) bereitstellen, um den lokalen Datenbestand synchron mit dem Jira-Board zu halten.